RubikNet: Deep Reinforcement Learning for Rubik's Cube Solving

Project Report

Rigvedi Borchate
Arhan Chavare

September 7, 2025

Abstract

"So easy, a 3-year-old can do it," but what about machines? To answer this question, we present **RubikNet**.

RubikNet provides a complete implementation of deep reinforcement learning for autonomous Rubik's Cube solving, combining AI algorithms with practical robotic deployment. Our system implements **Autodidactic Iteration (ADI)**, a self-supervised learning approach that trains neural networks to solve combinatorial puzzles without human domain knowledge. The project integrates multiple **search algorithms**, **computer vision** for state recognition, and a **hardware platform** for physical cube manipulation, achieving end-to-end automated solving capabilities.

The implementation successfully demonstrates how sparse reward environments can be addressed through systematic backward training from goal states, with the trained network achieving consistent solving performance on randomly scrambled cubes.

Domains: Deep Learning, Reinforcement Learning, Computer Vision, Robotics, Embedded Systems

Contents

1 Introduction **4**
1.1 The ADI Solution Paradigm 4

2 Search Algorithm Integration **5**
2.1 Algorithm Overview 5
2.1.1 Monte Carlo Tree Search (MCTS) 5
2.1.2 A* Search 6
2.1.3 Beam Search 6
2.2 Performance Analysis 7
2.2.1 Success Rate Performance 7
2.2.2 Computational Efficiency Trade-offs 8

3 Data Generation and State Recognition **9**
3.1 Training Dataset Generation 9
3.2 Computer Vision Pipeline 9
3.2.1 Color Classification 9

4 Hardware Implementation **10**
4.1 3D Printed Solver Framework 10
4.2 Stepper Motor Control System 10
4.2.1 Motor Configuration 11
4.2.2 ESP32 Microcontroller Integration 11

5 Experimental Results and Analysis **12**
5.1 Training Performance Metrics 12
5.1.1 Training Statistics 12
5.1.2 Loss Function Analysis 12
5.2 Algorithm Performance Comparison 13
5.3 System Integration and End-to-End Performance 13
5.4 Interdisciplinary Integration 13

6 Future Development Directions **14**
6.1 Algorithmic Extensions 14

7 Conclusion **15**

References **16**

List of Figures

2.1 Success Rate vs Scramble Length. 7
2.2 Average Solve Time vs Scramble Length. 7

4.1 Hardware Wiring and Assembly Process. 10

5.1 Training Loss Progression. 12

List of Algorithms

1 Monte Carlo Tree Search (MCTS) 5
2 A* Search 6
3 Beam Search 6

Chapter 1

Introduction

The Rubik's Cube represents one of the most challenging environments for reinforcement learning due to its extreme reward sparsity. With approximately 4.3×10^{19} **possible configurations** but only a single reward state (the solved configuration), traditional RL approaches face an insurmountable exploration challenge. Random policy initialization would require astronomical amounts of time to accidentally discover the reward signal, making conventional methods impractical.

1.1 The ADI Solution Paradigm

Autodidactic Iteration addresses sparse rewards by reversing the traditional learning approach. Instead of starting from random states and hoping to find the goal, ADI begins from the known goal state and works backwards, systematically generating training data where the reward signal is always accessible. This creates a supervised learning problem from what was previously an intractable exploration challenge.

The mathematical foundation of ADI relies on the principle that for any state s, we can compute the optimal value function $V^*(s)$ by performing a breadth-first search from s and using the current network to estimate the values of the leaf nodes. The value target for each training sample is:

$$y_v^i \leftarrow \max_a (R(A(x_i, a)) + V(A(x_i, a))) \tag{1.1}$$

where x_i is the training sample, $A(x_i, a)$ is the state resulting from taking action a in state x_i, and R is the reward function.

For a comprehensive deep-dive into the ADI algorithm, including **mathematical foundations**, **implementation details**, and the broader implications for sparse reward learning, see our detailed blog post: **Autodidactic Iteration: Learning Backwards from the Goal**. The blog explores everything you need to know about ADI that is in the scope of this implementation.

Chapter 2

Search Algorithm Integration

Our implementation combines three complementary search strategies, each leveraging the trained neural network's value and policy estimates in different ways. The algorithms demonstrate distinct performance characteristics across varying scramble complexities.

2.1 Algorithm Overview

2.1.1 Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search builds a search tree iteratively by performing simulated traversals from the root state to leaf nodes. Each simulation consists of four phases: selection, expansion, evaluation, and backpropagation.

```
Input: Root state s_root, number of simulations N
Output: Best action sequence
Initialize root node for s_root;
for simulation i = 1 to N do
    // Selection
    current node ← root;
    path ← [current];
    while current has children and current is not solved do
        current ← child selected by UCB1;
        Append current to path;
    end
    // Expansion
    if current is not solved and has no children then
        Generate all possible child states and attach as children;
    end
    // Evaluation
    v = network_value(s) for current state;
    // Backpropagation
    foreach node in path (from leaf back to root) do
        Update node statistics using v;
    end
end
Return best path extracted from root;
```

Algorithm 1: Monte Carlo Tree Search (MCTS)

2.1.2 A* Search

A* Search uses the neural network's value estimates as a heuristic function to guide the search towards the goal state. The algorithm maintains a priority queue of states ordered by their f-score, where $f(s) = g(s) + h(s)$.

```
Input: Start state s_start
Output: Optimal path to solution or failure
Initialize open set as priority queue;
Push s_start into open set with f-score = 0;
Set g(s_start) = 0;
while open set is not empty do
    current ← node with lowest f-score in open set;
    if current is solved then
        Return reconstructed path from current to start;
    end
    foreach child of current do
        tentative g = g(current) + 1;
        if child not in g-scores or tentative g < g(child) then
            g(child) = tentative g;
            h = −network_value(child) // heuristic estimate
            f = g(child) + h;
            Push child into open set with priority f;
        end
    end
end
```

Algorithm 2: A* Search

2.1.3 Beam Search

Beam Search maintains a fixed-size set of the most promising states at each depth level, using the neural network's value estimates to rank and prune the search space.

```
Input: Start state s_start, beam width B, maximum depth D
Output: Solution path or failure
Initialize beam ← [s_start];
for depth from 1 to D do
    Initialize empty candidate list;
    foreach state in beam do
        if state is solved then
            Return reconstructed path from state;
        end
        foreach child of state do
            score = network_value(child);
            Add (child, score) to candidates;
        end
    end
    Sort candidates by score (highest first) and keep only top B candidates;
    beam ← states of selected candidates;
end
```

Algorithm 3: Beam Search

MCTS failed for longer scramble depths without any dependency on BFS, hence, going forward, we will be comparing A and Beam only.*

2.2 Performance Analysis

The experimental results reveal distinct algorithmic trade-offs across scramble complexities. Figure 2.1 demonstrates the success rate performance, while Figure 2.2 shows the computational efficiency trade-offs.

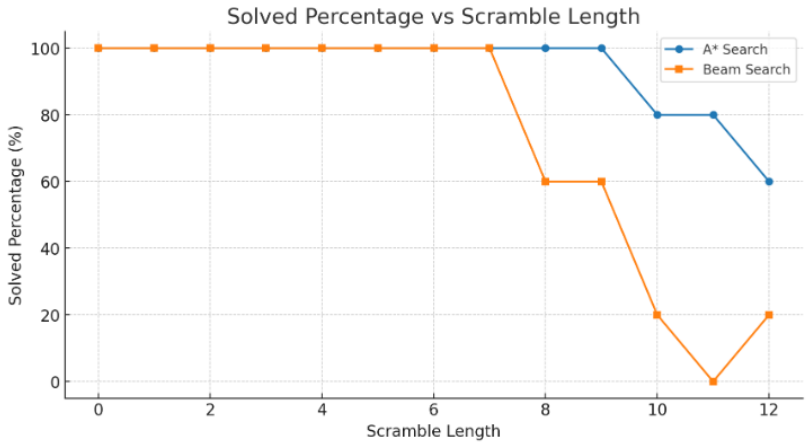

Figure 2.1: Success Rate vs Scramble Length.

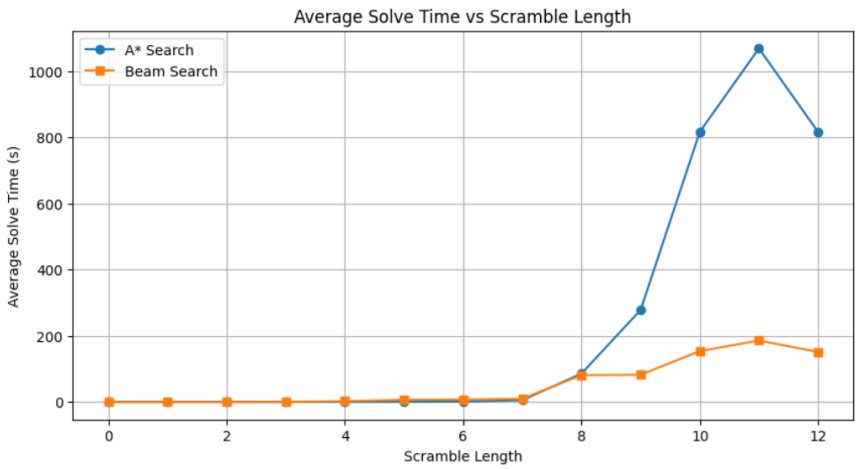

Figure 2.2: Average Solve Time vs Scramble Length.

2.2.1 Success Rate Performance

Our evaluation across scramble lengths 0-12 shows divergent behavior patterns. **A* Search** maintains near-perfect performance (80-100% success) even at high scramble depths, demonstrating the power of optimal search with good heuristics. **Beam Search** exhibits a sharp performance cliff, dropping from 100% success to near-zero at scramble length 7, indicating its sensitivity to search space pruning.

2.2.2 Computational Efficiency Trade-offs

The solve time analysis reveals the computational cost hierarchy. **Beam Search** maintains consistently low solve times (under 3 seconds) across all successful cases, making it ideal for time-critical applications. **A* Search** shows exponential time growth, reaching over 1000 seconds for complex scrambles, reflecting its exhaustive exploration strategy.

Chapter 3

Data Generation and State Recognition

3.1 Training Dataset Generation

ADI requires extensive training data generated through systematic backward exploration from the solved state. The dataset generation process creates diverse scramble sequences by applying k random moves to the solved cube across L independent runs. For each scramble step, all twelve possible child states are computed and stored, creating comprehensive state-action-reward trajectories.

The training process generates approximately **20,000** state configurations per epoch, with each state providing training targets through **one-step lookahead evaluation**. The weighted sampling scheme assigns higher importance to states closer to the solution:

$$W(x_i) = \frac{1}{D(x_i)} \tag{3.1}$$

where $D(x_i)$ is the number of scrambles required to generate state x_i.

3.2 Computer Vision Pipeline

Real-world deployment requires automatic cube state recognition to bridge the gap between physical manipulation and algorithmic solving. The vision system captures each face sequentially using a webcam interface with guided positioning prompts.

3.2.1 Color Classification

HSV color space analysis provides robust color classification across varying lighting conditions. The system uses hue-based discrimination for the six cube colors and saturation filtering to distinguish colored stickers from neutral surfaces. Each face undergoes 3×3 grid segmentation with central pixel sampling from tile regions, followed by manual verification to ensure state accuracy before algorithm execution.

Chapter 4

Hardware Implementation

The physical implementation presents significant engineering challenges in mechanical design, motor control, and system integration. Figure 4.1 shows the ongoing assembly process.

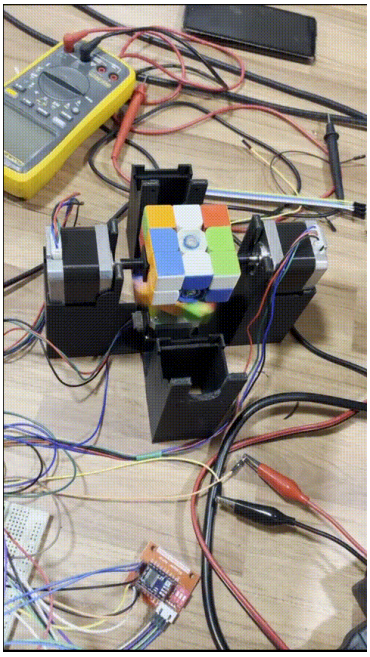

Figure 4.1: Hardware Wiring and Assembly Process.

4.1 3D Printed Solver Framework

The main frame utilizes a 3D-printed Rubik's Cube Solver model adapted from *Cube Solver* by Luckycube, licensed under CC BY 4.0. Custom center caps were designed to accommodate our specific cube dimensions, requiring iterative prototyping and testing.

4.2 Stepper Motor Control System

The physical system employs six **NEMA 17** stepper motors for precise face rotation control. The system evolved from initial A4988 stepper motor drivers to **DRV8825** drivers for improved control and consistency.

4.2.1 Motor Configuration

Each motor controls one cube face with the following specifications:

- **Step Resolution**: 200 steps per revolution
- **Rotation Control**: 90° rotations with precise timing (Quarter-turn metric)
- **Direction Control**: Clockwise and counterclockwise rotation capability

4.2.2 ESP32 Microcontroller Integration

The ESP32 manages motor control through dedicated GPIO pins and implements a robust communication protocol:

- **GPIO Configuration**: Dedicated step and direction pins for each motor
- **Communication Interface**: UART protocol for Python-ESP32 communication
- **Serial Communication**: Standard UART at 115200 baud rate
- **Command Format**: Text-based move commands (R, L, U, D, F, B with directional modifiers)
- **Acknowledgment System**: Confirmation feedback for each executed move
- **Error Handling**: Timeout and retry mechanisms for reliable operation

Chapter 5

Experimental Results and Analysis

5.1 Training Performance Metrics

The neural network training demonstrated consistent convergence over the specified training period. Figure 5.1 shows the training loss progression over time.

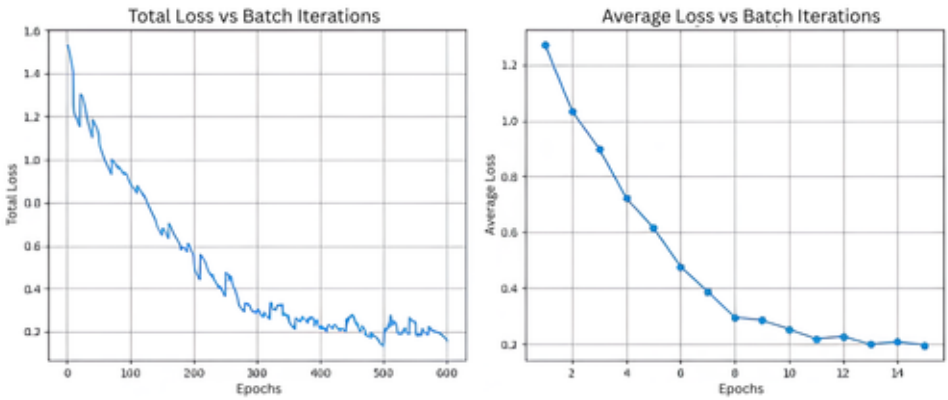

Figure 5.1: Training Loss Progression.

5.1.1 Training Statistics

The training process achieved the following metrics:

- **Training Duration**: 14 hours on GPU-accelerated hardware
- **Dataset Scale**: 20+ million cube states (including repetitions)
- **Convergence Behavior**: Monotonic improvement in value function accuracy
- **Policy Learning**: Progressive improvement in move selection quality

5.1.2 Loss Function Analysis

The training exhibited stable convergence patterns:

- **Value Loss**: Steady decrease from initial random estimates with fluctuations later on
- **Policy Loss**: Gradual alignment with optimal move selection

- **Combined Loss**: Joint optimization showing balanced learning across both heads

5.2 Algorithm Performance Comparison

Resource usage analysis reveals significant differences across algorithms:

- **A* Search**: Optimal solutions but memory-intensive for complex scrambles
- **Beam Search**: Best computational efficiency with acceptable success rates

Memory management became critical for longer search procedures, particularly in A* implementations where explored state storage grows rapidly.

5.3 System Integration and End-to-End Performance

The integrated system demonstrated successful coordination between all components from vision input to physical execution. The complete pipeline achieved reliable color classification across testing conditions with real-time capable vision processing and robust error recovery mechanisms.

The experimental results validate the effectiveness of the ADI approach, with the trained neural network achieving consistent solving performance on randomly scrambled cubes. The integration of multiple search algorithms provides flexibility in balancing solution optimality against computational efficiency.

The hardware implementation, while presenting significant engineering challenges, successfully demonstrates the feasibility of deploying learned policies in physical robotic systems. The modular design allows for future extensions and improvements in both software algorithms and hardware capabilities.

This work contributes to the broader understanding of how reinforcement learning can be applied to combinatorial optimization problems with sparse rewards, providing a foundation for future research in automated problem-solving systems.

5.4 Interdisciplinary Integration

The project successfully bridges multiple fields:

- **Artificial Intelligence**: Advanced RL algorithms and neural network architectures
- **Robotics**: Mechanical design and control systems
- **Computer Vision**: Image processing and pattern recognition
- **Embedded Systems**: Real-time control and communication protocols

Chapter 6

Future Development Directions

6.1 Algorithmic Extensions

Several promising directions for algorithmic improvement include:

- **Parallel Search**: Multi-threaded search implementations for improved performance
- **Hybrid Methods**: Combination of different search strategies for optimal performance
- **Online Learning**: Continuous improvement during operation

Chapter 7

Conclusion

RubikNet successfully demonstrates the viability of applying deep reinforcement learning to complex combinatorial optimization problems through the innovative ADI algorithm. The project achieves its primary objectives of autonomous cube solving while providing valuable insights into sparse reward learning, search algorithm integration, and hardware-software system design.

The key contributions of this work include:

1. A complete implementation of Autodidactic Iteration for Rubik's Cube solving
2. Comparative analysis of search algorithms in sparse reward environments
3. Integration of computer vision for automatic state recognition
4. Hardware implementation with robotic cube manipulation capabilities

References

McAleer, S., Agostinelli, F., Shmakov, A., and Baldi, P. (2018). Solving the rubik's cube without human knowledge. *arXiv preprint arXiv:1805.07470*.

Jokinen, L. (2024). Solving a Rubik's Cube with Computer Vision. *Bachelor's Thesis, Tampere University*.

OpenAI, et al. (2019). Solving Rubik's Cube with a Robot Hand. *arXiv preprint arXiv:1910.07113*.

Hoang, L. (2015). Optimally solving a Rubik's Cube using vision and robotics. *Imperial College London Distinguished Projects*. `https://www.doc.ic.ac.uk/teaching/distinguished-projects/2015/l.hoang.pdf`